

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	S.GAYATHERI	Reg. No.:	192521306
Date:	03/08/2026	Duration:	60 minutes

Code of Conduct

I S.GAYATHERI (192521306) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.GAYATHERI

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	Mark
System Requirement Analysis	/5
Selection of Software Architecture	/5
Architecture Diagram and Component Design	/5
Component Interaction and Quality Attribute Analysis	/5
Architectural Justification and Technical Presentation	/5
TOTAL	/5

1.1 System Objectives

The AI-Powered Personalized Nutrition and Health Recommendation Platform is designed to provide customized nutrition guidance and health recommendations by analyzing user health information, dietary preferences, fitness goals, and wearable device data. The platform leverages Artificial Intelligence (AI) to generate personalized meal plans, monitor nutrition intake, and provide continuous health insights to improve users' overall well-being.

Objectives

- Collect personal health and lifestyle information.
- Analyze nutritional requirements using AI techniques.
- Generate personalized meal plans based on health conditions and goals.
- Monitor users' daily dietary intake.
- Integrate wearable devices to collect health metrics.
- Provide intelligent health and nutrition recommendations.
- Track nutrition progress and health improvements.
- Encourage healthy lifestyle adoption.

1.2 Functional Requirements

User Management

- User registration and login.
- Profile management.
- Secure authentication and authorization.

Health Information Management

- Record age, gender, height, weight, BMI.
- Store medical history and allergies.
- Capture dietary preferences and fitness goals.

AI Nutrition Analysis

- Analyze nutritional deficiencies.

- Calculate calorie and nutrient requirements.
- Generate personalized dietary recommendations.

Meal Planning

- Recommend breakfast, lunch, dinner, and snacks.
- Suggest healthy food alternatives.
- Recommend daily water intake.

Wearable Device Integration

- Synchronize fitness data from Fitbit, Google Fit, Apple Health, and other devices.
- Monitor heart rate, calories burned, steps, and sleep quality.

Progress Monitoring

- Track daily calorie intake.
- Monitor BMI and weight changes.
- Generate weekly and monthly nutrition reports.

Notification System

- Meal reminders.
- Water intake reminders.
- Exercise and health alerts.

Administration

- Manage users.
- Update nutrition database.
- Monitor system usage and reports.

1.3 Non-Functional Requirements

Performance

- Response time should be less than 3 seconds.
- Real-time AI recommendations.

Security

- User authentication using JWT/OAuth.
- Data encryption.

- HTTPS communication.
- Role-based access control.

Reliability

- Continuous operation.
- Automatic backup and recovery.

Scalability

- Support thousands to millions of users.
- Cloud-based deployment.

Maintainability

- Modular software design.
- Easy testing and updates.

Availability

- High system uptime (24×7).
- Cloud redundancy and failover.

Usability

- Responsive web and mobile interface.
- Easy-to-use dashboard.

1.4 Quality Attributes

Quality Attribute	Description
Scalability	Supports increasing users and wearable devices through independent service scaling.
Security	Protects sensitive health data using authentication, encryption, and secure APIs.
Performance	Provides fast AI recommendations with optimized processing and caching.
Reliability	Ensures continuous operation with fault isolation and backups.
Maintainability	Modular services simplify updates, testing, and debugging.
Availability	Cloud deployment ensures high uptime and disaster recovery.

2. Software Architecture Selection

Selected Architecture

Hybrid Architecture (Microservices + Layered Architecture)

Why Hybrid Architecture?

The proposed system combines Microservices Architecture with Layered Architecture to achieve scalability, flexibility, and maintainability.

- Microservices divide the application into independent services such as AI Recommendation, User Management, Meal Planning, and Notifications. Each service can be developed, deployed, and scaled independently.
- Layered Architecture organizes each service into Presentation, Business Logic, and Data Access layers, improving code organization and maintainability.

Justification

- Dedicated AI service for intelligent nutrition recommendations.
- Modular design simplifies maintenance and testing.
- Independent deployment of services.
- High scalability for growing users.
- Secure communication through API Gateway.
- Easy integration with wearable devices and third-party healthcare APIs.
- Supports future enhancements such as telemedicine and predictive analytics.

3. Architectural Decisions

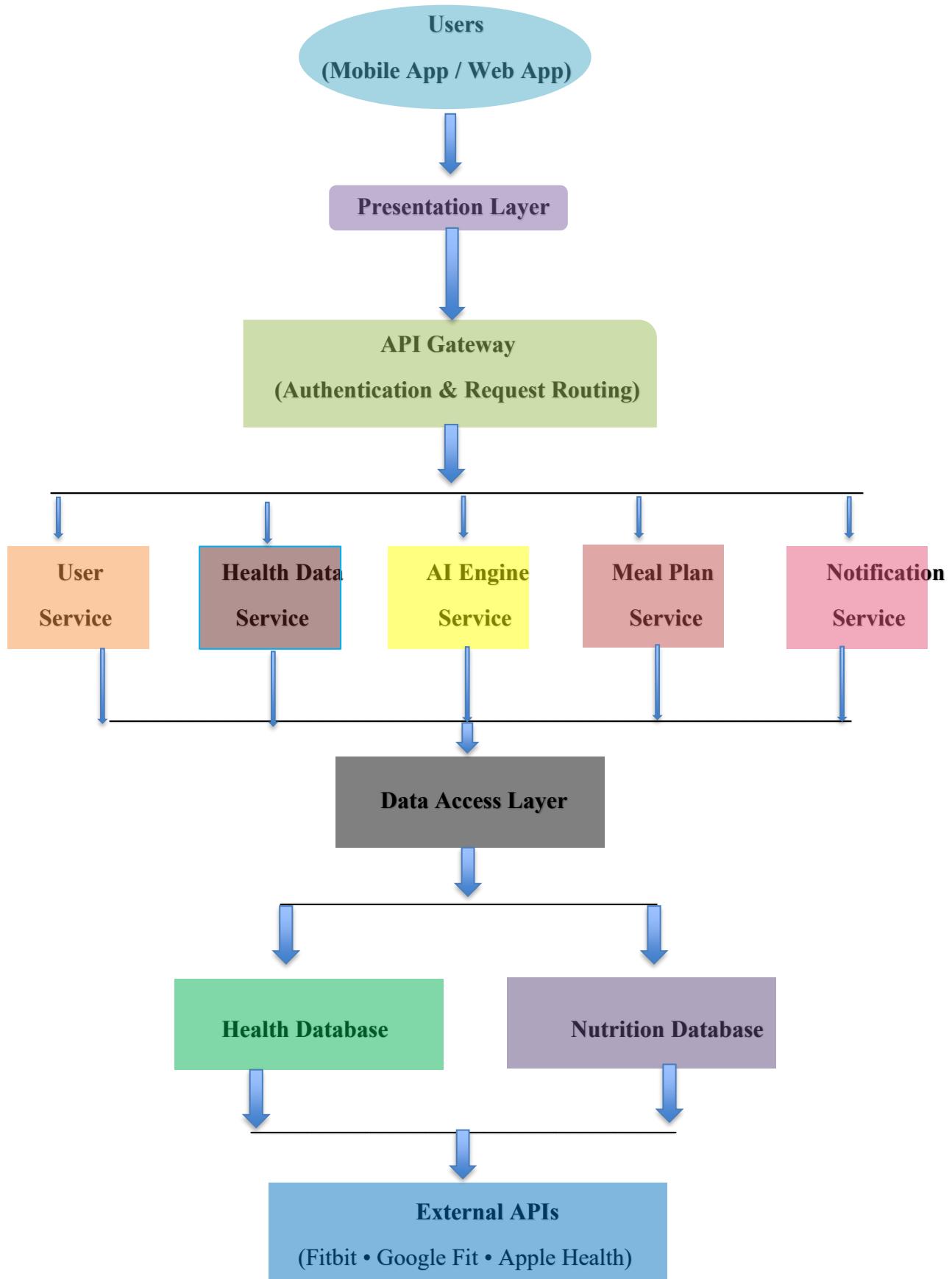
Rationale Behind Choosing Hybrid Architecture

The Hybrid Architecture (Microservices + Layered) was selected because it provides the flexibility, scalability, and modularity required for an AI-driven healthcare platform.

Reasons

- Supports AI-based nutrition recommendation processing.
- Allows independent deployment and scaling of individual services.
- Simplifies maintenance through modular service design.
- Enables secure communication using an API Gateway.

3. Software Architecture Diagram



4. Components and Their Interactions

Components and Responsibilities

Component	Responsibility
Presentation Layer	Provides user interface through mobile and web applications.
API Gateway	Authenticates users, routes requests, and secures communication between clients and services.
User Service	Handles user registration, login, profile management, and authentication.
Health Data Service	Stores and manages personal health records, medical history, allergies, and fitness goals.
AI Recommendation Service	Analyzes health data using AI algorithms and generates personalized nutrition recommendations.
Meal Planning Service	Creates customized meal plans based on AI recommendations and nutritional requirements.
Notification Service	Sends reminders for meals, hydration, medication, and exercise.
Health Database	Stores user profiles, health records, reports, and AI recommendations.
Nutrition Database	Stores food information, calorie values, nutrients, and meal templates.
External APIs	Integrates wearable devices such as Fitbit, Google Fit, and Apple Health to collect activity data.

Component Interaction Workflow

1. The user accesses the system through the mobile or web application.
2. The API Gateway authenticates the user and routes requests.
3. The User Service retrieves or updates profile information.
4. The Health Data Service collects medical history, health parameters, and fitness goals.

5. The Wearable APIs provide activity data such as steps, calories burned, heart rate, and sleep patterns.
6. The AI Recommendation Service analyzes all collected information.
7. The Meal Planning Service generates personalized meal plans and dietary suggestions.

5. Quality Attribute Evaluation

Quality Attribute	Implementation
Scalability	Each microservice (AI, Meal Planning, Notifications, User Management) can scale independently to handle increasing workloads.
Security	JWT authentication, HTTPS communication, encrypted databases, role-based access control, and secure API Gateway protect sensitive health information.
Performance	Dedicated AI services, optimized database queries, and caching mechanisms reduce response time for nutrition recommendations.
Reliability	Microservices isolate failures, while automatic backups and cloud recovery improve system reliability.
Maintainability	Layered architecture organizes business logic, making services easier to modify, test, and maintain.
Availability	Cloud deployment, replicated databases, load balancing, and failover mechanisms ensure high availability and continuous operation.

Trade-offs Considered

Advantages	Trade-offs
Independent service deployment	More complex deployment and orchestration
Better scalability	Increased inter-service communication
Easier maintenance	Requires API Gateway and service discovery

Advantages	Trade-offs
Fault isolation	Additional monitoring and logging infrastructure
Future extensibility	Higher initial development effort

Future Enhancements and Long-Term Maintainability

The chosen architecture supports future expansion by allowing new services to be added independently without modifying the entire system.

Possible future enhancements include:

- Telemedicine consultation integration.
- Predictive health risk analysis using advanced AI models.
- Smart grocery and shopping recommendations.
- Voice assistant integration.
- Electronic Health Record (EHR) connectivity.
- Hospital and pharmacy system integration.
- Advanced analytics and personalized wellness coaching.

Because each function is implemented as an independent microservice with a layered internal design, the platform remains scalable, maintainable, and adaptable to evolving healthcare technologies.

Conclusion

The Hybrid Architecture (Microservices + Layered Architecture) is the most appropriate architectural style for the AI-Powered Personalized Nutrition and Health Recommendation Platform. It fulfills the project's functional and non-functional requirements by providing a scalable, secure, reliable, high-performance, and maintainable solution. The architecture supports AI-driven nutrition analysis, wearable device integration, cloud-based data management, and continuous health monitoring while also providing a strong foundation for future enhancements and long-term system evolution.